IBM MACHINE LEARNING CAPSTONE

Personalized Online Course Recommender System

Exploring content-based, clustering, collaborative-filtering, and predictive recommendation methods

Mubashir Ali | November 2026

Why a course recommender?

Online catalogs contain hundreds of courses. A useful recommender should turn sparse enrollment history and course metadata into relevant next steps for each learner.

307
courses in genre catalog

233,306
course interactions

33,901
unique learners

126
rated course items

Hypothesis 1: course genres and text similarity can identify relevant alternatives.
Hypothesis 2: learner–course interactions reveal complementary preferences.
Hypothesis 3: combining multiple model families gives a stronger decision basis.

Data and analytical workflow

01

Explore

course genres and enrollment patterns

→

02

Represent

genre vectors, BoW and embeddings

→

03

Recommend

similarity, clusters and CF

→

04

Evaluate

ranking coverage, RMSE and accuracy

Implemented notebook surfaces

EDA and feature engineering

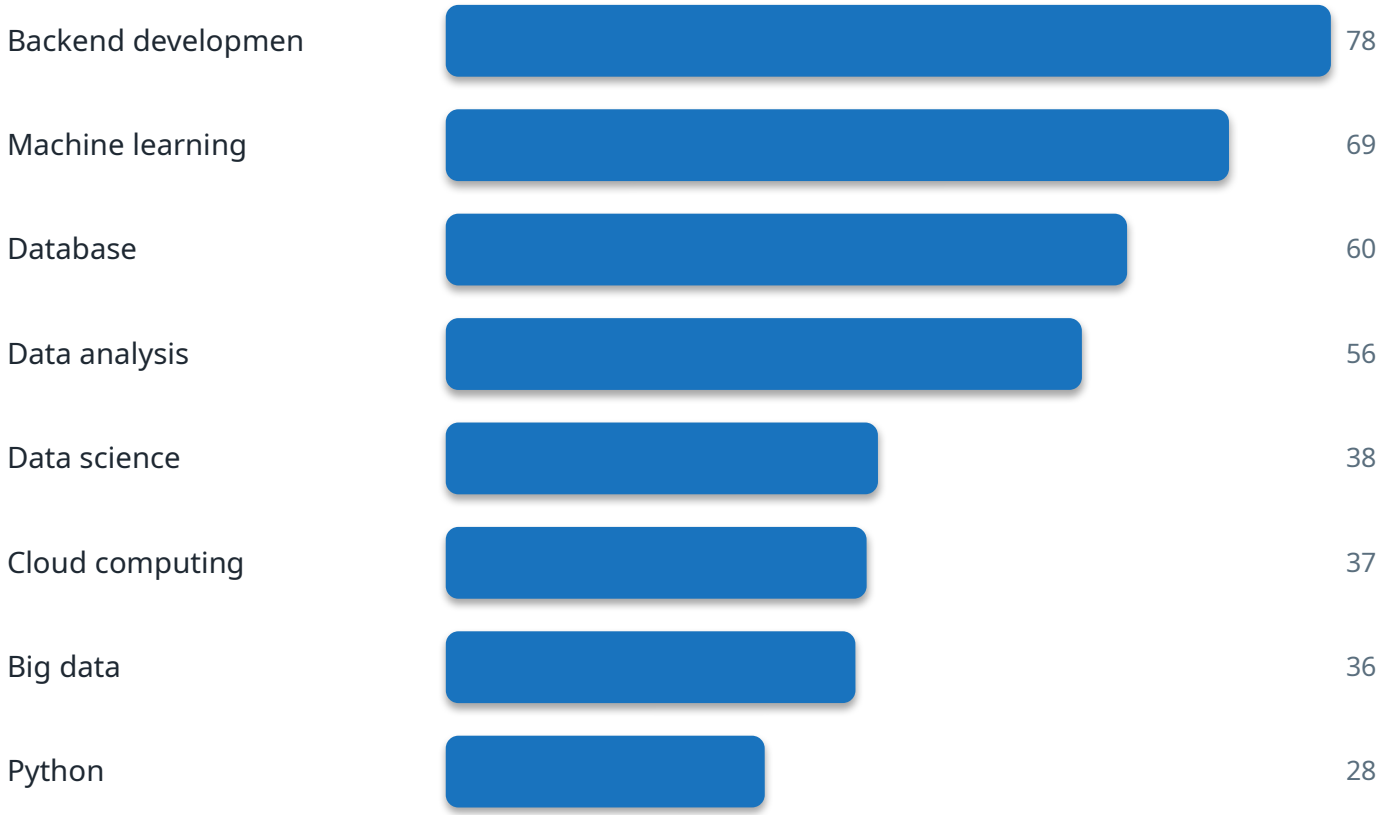
User-profile and course-similarity recommenders

K-means clustering, KNN, NMF and neural embeddings

Regression and classification rating prediction

Exploratory data analysis: the catalog

The genre catalog is broad, but concentrated in technical and data-oriented subjects.



Feature engineering

Binary genre indicators encode course topics.

Course titles and descriptions are transformed into sparse BoW vectors.

Weighted genre profiles summarize each learner's interests.

Source: course_genre.csv (307 rows)

Exploratory data analysis: demand signals

Enrollment activity is highly uneven: a small set of foundational courses dominates interactions.



Implication

Popularity is a strong baseline, but it can over-recommend common introductory courses.

Personalization is needed to surface less obvious, high-fit alternatives.

Source: ratings.csv; counts are interaction rows by item.

Content-based recommendation

Learner profile + course genres

1. Aggregate a learner's enrolled-course genres into a weighted profile.
2. Compare the profile with candidate course genre vectors.
3. Rank unseen courses by cosine similarity / recommendation score.

Two complementary content signals

Genre profile

Interpretable interests such as Python, Machine Learning and Data Analysis.

BoW similarity

Text-level similarity captures related course titles and descriptions.

Similarity and clustering recommenders

Course similarity

Build a course-by-course similarity matrix from BoW features.

For each learner, combine recommendations from enrolled courses.

Filter already-seen courses and rank by similarity.

Best suited for: “more like this” discovery

User-profile clustering

Reduce high-dimensional profiles with PCA.

Group learners with K-means.

Recommend popular unseen courses within the learner’s cluster.

Best suited for: cohort-level personalization

Collaborative filtering models

Interaction history supports recommendations based on collective behavior rather than course metadata.

KNN

Neighborhood-based similar learners/items

NMF

Latent factors from user–item matrix

Neural network

Learned user and course embeddings

Key trade-off Collaborative filtering can discover unexpected matches, but cold-start learners and sparse histories remain challenging.

Predictive rating models

Regression

Predict a continuous rating score from user and course embedding features.

Evaluation: RMSE / MAE

Classification

Predict a rating mode or class from the same embedding-based representation.

Evaluation: accuracy / F1

These models make the recommendation problem measurable as supervised prediction.

Evaluation plan

Each model family answers a different product question; evaluation should reflect the intended use.

Model family	Primary question	Useful metrics
Content / clustering	Are recommendations relevant and novel?	Recall@K, coverage, unseen courses
KNN / NMF	Can we predict held-out interactions?	RMSE, MAE, ranking metrics
Regression	How close is the predicted score?	RMSE, MAE
Classification	Can we identify the rating mode?	Accuracy, precision, recall, F1

Current repository status: notebooks contain the modeling workflow, but saved evaluation outputs are not included.

From notebook to learner experience

A Streamlit interface can make the models inspectable and useful without hiding their assumptions.

1 Select a course

Choose an enrolled or seed course.

2 Choose a strategy

Similarity, profile, cluster or CF.

3 Review results

See ranked courses and rationale.

Recommended explanation fields

similar genres • similar course text • cluster popularity • predicted rating

Conclusions and next steps

What this project establishes

A full recommender-system workflow from EDA through model families.

Multiple representations: genres, BoW, interaction matrices and embeddings.

A principled basis for comparing relevance, novelty and prediction quality.

Next steps

Extend the holdout evaluation to ranking and novelty metrics.

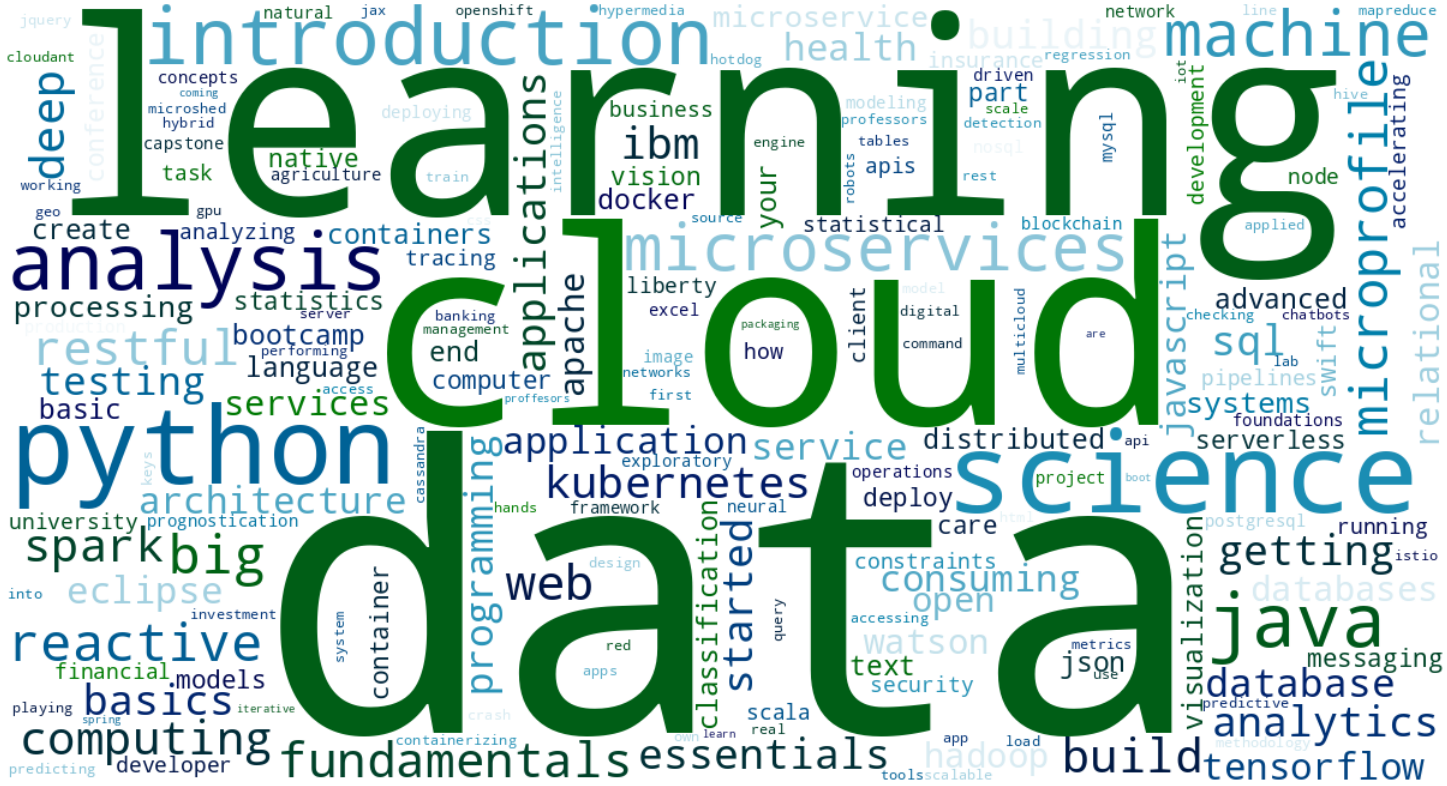
Tune thresholds and K values using held-out users/interactions.

Build and deploy the Streamlit demo with transparent explanations.

Thank you

Course-title word cloud

After lowercasing, tokenization and removal of generic stopwords, the catalog is dominated by data and cloud vocabulary.



Prominent terms

data (89)
learning (32)
cloud (32)
science (30)
python (27)
analysis (27)

Source: 307 course titles; counts are post-preprocessing token frequencies.

Collaborative-filtering workflows

KNN

Build an item-item similarity matrix from the training user-item matrix. For a held-out item, combine the learner's ratings on its nearest neighbors using similarity-weighted averaging.

Input → representation → score

NMF

Factorize the non-negative user-item matrix into learner factors W and course factors H. Estimate a missing rating with the dot product W[u] · H[:, i].

Input → representation → score

Neural embeddings

Map user and course IDs to learned vectors, combine the embeddings in a neural network, and output a predicted rating score.

Input → representation → score

Measured collaborative-filtering comparison

Reproducible holdout experiment: 80/20 random split, 186,644 train interactions, 46,662 test interactions, random_state=42.

Model	RMSE	MAE	Interpretation
Item-item KNN (k=20)	0.894	0.760	Best of the tested baselines
NMF (15 factors)	2.958	2.811	Weak with zero-filled sparse matrix

Finding KNN is the strongest measured baseline here. NMF needs missing-data-aware training or stronger regularization before it is a fair production candidate.

Metrics are rating-prediction metrics; they do not replace ranking, coverage or novelty evaluation.

Evaluation results for recommendation quality

The next evaluation layer should measure the recommender as a ranking system, not only as a rating predictor.

Coverage

How much catalog?

Fraction of courses ever recommended.

Novelty

How surprising?

Share of unseen or long-tail courses.

Recall@K

How relevant?

Held-out courses recovered in top K.

Recommended reporting: compare these metrics across user-profile, course-similarity and clustering recommenders at the same K.